code to add a `staging` profile, and assign it to the beans marked as `prod` (which prevents your application from being immutable and deployable to any environment without any change to the source code). Instead, I recommend using profiles as feature flags when they're associated with groups of beans to be loaded conditionally. Consider what functionality a profile provides, and name it accordingly, rather than thinking about where it will be enabled.

You might still have cases where a bean that handles infrastructural concerns is required in specific platforms. For example, you might have certain beans that should only be loaded when the application is deployed to a Kubernetes environment (no matter whether it is for staging or production). In that case, you could define a `kubernetes` profile.

In chapter 3 we built the Catalog Service application to manage books. Whenever you run it locally, there are no books in the catalog yet, and you need to add some explicitly if you want to work with the application. A better option would be to let the application generate some test data at startup, but only when it's needed (for example, in a development or test environment). Loading test data can be modeled as a feature that you enable or disable through configuration. You could define a `testdata` profile to toggle the loading of this test data. In that way, you'll keep the profiles independent from the deployment environment, and you can use them as feature flags with no constraint on the deployment environment whatsoever. Let's do that.

First, add a new `com.polarbookshop.catalogservice.demo` package to your Catalog Service project, and create a `BookDataLoader` class. You can instruct Spring to load this class only when the `testdata` profile is active by applying the `@Profile` annotation. Then you can use the